《计算机图形学及人机交互》

实验报告

学　号：2023112621

姓　名：程泽凯

年　级：2023 级

专　业：计算机科学与技术

院　系：计算机与人工智能学院

2025 2026 第 1 学期

目录

1 实验目的、任务及要求 2
1.1 实验目的 . . . 2
1.2 实验任务与要求 . . . 2

2 实验环境 3
2.1 硬件环境 . . . 3
2.2 软件环境 . . . 3

3 主要功能模块结构及说明 4
3.1 系统总体架构 . . . 4
3.2 核心模块详细说明 . . . 4
3.2.1 直线绘制模块 . . . 4
3.2.2 圆绘制模块 . . . 5
3.2.3 线宽修改模块 . . . 6
3.2.4 线型修改模块 . . . 7
3.2.5 区域填充模块 . . . 8
3.2.6 Bezier 曲线模块 . . . 8

4 实验算法及程序流程 10
4.1 直线绘制算法 . . . 10
4.1.1 中点画线法 . . . 10
4.1.2 Bresenham 画线法 . . . 11
4.2 圆绘制算法 . . . 13
4.2.1 中点画圆法 . . . 13
4.2.2 Bresenham 画圆法 . . . 14
4.3 种子填充算法 . . . 15
4.4 扫描线填充算法 . . . 17
4.5 粗线绘制算法 . . . 19
4.6 粗圆绘制算法 . . . 20
4.7 Bezier 曲线算法 . . . 23
4.7.1 de Casteljau 递推算法 . . . 23
4.7.2 Bezier 曲线生成流程 . . . 24

5 实验结果 26

6 实验总结 26

1 实验目的、任务及要求

1.1 实验目的

通过本次实验，学习并实现计算机图形学中的基本图元生成算法，掌握区域填充技术和曲线绘制方法。

1. 掌握中点画线法和 Bresenham 画线法的原理与实现
2. 掌握中点画圆法和 Bresenham 画圆法的原理与实现
3. 理解并实现种子填充算法和扫描线填充算法
4. 掌握 Bezier 曲线的 de Casteljau 递推算法
5. 学习图形属性（线型、线宽）的控制方法

1.2 实验任务与要求

1. 直线绘制算法：采用中点画线法和 Bresenham 法实现直线段的绘制
2. 圆绘制算法：采用中点画圆法和 Bresenham 法实现圆的绘制
3. 线宽修改：采用扫描线填充法修改直线和圆的线宽
4. 线型修改：实现实线、虚线、点线、点划线
5. 区域填充：采用种子填充法和扫描线填充法对封闭区域进行填充
6. **Bezier** 曲线：采用 de Casteljau 递推算法绘制 Bezier 曲线

2 实验环境

2.1 硬件环境

1. 计算机：机械革命旷世 16pro
2. CPU：12th Gen Intel® i9-12900H
3. RAM：32GB
4. 显卡：NVIDIA GeForce RTX 4070 Laptop GPU

2.2 软件环境

1. 操作系统：Windows 11 专业版
2. 开发工具：Visual Studio 2022
3. 编程语言：C++17
4. 图形 API：Direct2D (Windows SDK)
5. 项目类型：Win32 桌面应用程序

3 主要功能模块结构及说明

3.1 系统总体架构

本系统基于实验 2 的绘图软件扩展，采用模块化设计。根据实验任务要求，系统主要包括直线绘制、圆绘制、线宽修改、线型修改、区域填充和 Bezier 曲线六大功能模块。

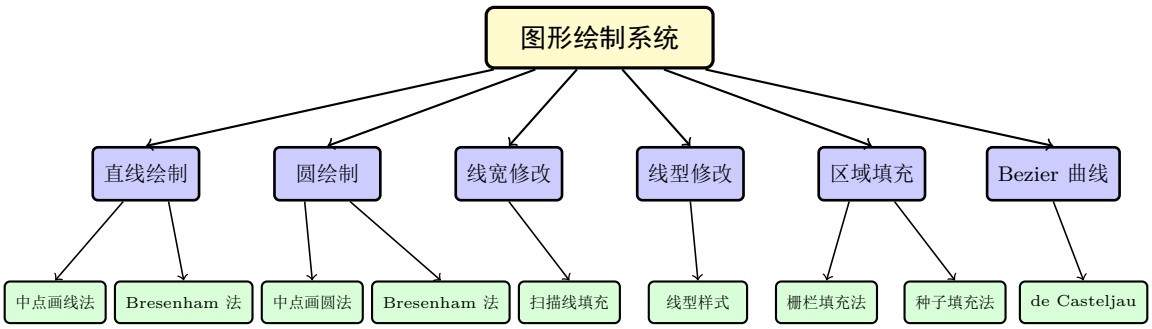


图 1: 系统模块结构图

3.2 核心模块详细说明

3.2.1 直线绘制模块

模块功能：
本模块负责实现直线段的光栅化绘制，是图形系统中最基础的图元绘制功能。直线绘制是所有图形系统的核心，其他复杂图形（如多边形、曲线）都依赖于直线的绘制。模块提供了两种经典的直线绘制算法，用户可以通过菜单选择使用中点画线法或 Bresenham 画线法，两种算法各有特点但效果相同。
主要功能：

- 任意方向直线：支持任意起点和终点的直线绘制，自动处理 8 个方向（水平、垂直、斜向）
- 整数运算：完全使用整数运算，避免浮点误差，保证绘制精度
- 连续性保证：生成连续的像素点序列，保证直线的连通性，无断点
- 算法切换：支持中点法和 Bresenham 法实时切换，便于算法对比学习
- 模块协作：输出标准像素点坐标，可与线宽、线型模块无缝配合
- 性能优化：采用增量算法，每步只需简单加减运算，效率极高

用户交互：

- 通过” 直线算法” 菜单选择中点法或 Bresenham 法，菜单显示当前算法
- 在画布上点击鼠标左键确定起点，拖动鼠标到目标位置
- 拖动过程中实时预览直线效果（橡皮筋技术）
- 释放鼠标左键完成绘制，直线固定在画布上
- 支持撤销/重做操作，可删除或修改已绘制的直线

技术特点：

- 两种算法本质相同，都基于增量决策参数
- 完全使用整数运算，效率高
- 通过坐标变换统一处理 8 个象限
- 输出像素点坐标，便于后续处理

3.2.2 圆绘制模块

模块功能：
本模块实现圆的光栅化绘制，是图形系统中另一个重要的基本图元。圆形在图形设计、工程制图、游戏开发中应用广泛。模块通过利用圆的八对称性特点，将计算量降低到原来的 1/8，大幅提高绘制效率。提供中点画圆法和 Bresenham 画圆法两种经典算法供用户选择和对比学习。
主要功能：

- 圆形绘制：根据圆心坐标和半径值绘制完整的圆形
- 八对称性优化：利用圆的对称性，只需计算 1/8 圆弧（45 度）即可得到完整圆形，效率提升 8 倍
- 任意半径：支持从 1 像素到屏幕尺寸的任意半径圆形绘制
- 均匀分布：生成均匀分布的圆周像素点，避免出现断点或聚集
- 样式支持：可与线宽、线型模块配合，实现粗圆、虚线圆等不同样式
- 算法对比：支持两种算法切换，便于学习和性能对比

用户交互：

- 通过” 圆算法” 菜单选择中点法或 Bresenham 法

- 在画布上点击鼠标左键确定圆心位置
- 拖动鼠标，圆的半径随鼠标距离实时变化
- 实时显示圆的预览效果，可精确控制圆的大小
- 释放鼠标完成绘制，支持撤销/重做操作

技术特点：

- 八对称性使效率提升 8 倍，只需计算 45 度圆弧
- 完全整数运算，无浮点误差
- 通过增量决策参数判断下一个像素位置
- 生成的圆形像素分布均匀，视觉效果好

3.2.3 线宽修改模块

模块功能：
本模块实现直线和圆的线宽控制功能，采用扫描线填充法将细线转换为具有指定宽度的粗线。这是实验要求的核心功能之一，体现了扫描线填充算法的实际应用。
主要功能：

- 支持 1-20 像素的线宽设置
- 粗线绘制：将线段转换为矩形区域并填充
- 粗圆绘制：将圆转换为环形区域并填充
- 自动处理线宽为 1 时的特殊情况（直接绘制细线）
- 保持线段端点和圆形的几何精度

用户交互：

- 通过” 线宽” 菜单选择 1、3、5、10 等预设线宽
- 也可通过对话框输入自定义线宽值
- 线宽设置对后续绘制的所有图形生效
- 可实时看到不同线宽的效果

技术特点：

- 粗线通过计算垂直方向向量构造矩形
- 粗圆通过内外圆半径差构造环形
- 使用 Direct2D 的 PathGeometry 和 FillGeometry 实现高效填充
- 支持 GPU 硬件加速，性能优异

3.2.4 线型修改模块

模块功能：
本模块实现图形的线型样式控制，提供实线、虚线、点线、点划线四种常用线型。通过控制像素的绘制模式，实现不同的视觉效果。
主要功能：

- 实线（SOLID）：连续绘制所有像素，默认线型
- 虚线（DASH）：长短相间的虚线效果，适合表示辅助线
- 点线（DOT）：点状线型，适合表示参考线
- 点划线（DASHDOT）：点划结合的线型，适合表示中心线
- 线型设置对所有图形（直线、圆、矩形等）生效

用户交互：

- 通过” 线型” 菜单选择四种线型之一
- 线型设置立即应用到后续绘制的图形
- 可与线宽设置组合使用，实现多样化效果
- 菜单中显示当前选中的线型

技术特点：

- 算法级实现：根据像素索引和模式数组控制绘制
- Direct2D 实现：使用 ID2D1StrokeStyle 接口设置线型
- 模式数组定义灵活，易于扩展新线型
- 与硬件加速兼容，性能不受影响

3.2.5 区域填充模块

模块功能：
本模块实现封闭区域的颜色填充功能，提供栅栏填充法（扫描线填充）和种子填充法两种算法，满足不同场景的填充需求。
主要功能：

- 栅栏填充法：适用于已知顶点的多边形填充（矩形、三角形、多边形等）
- 种子填充法：适用于任意形状的封闭区域填充，用户点击区域内部即可填充
- 支持自定义填充颜色
- 自动识别封闭区域边界
- 处理复杂多边形（凸多边形和凹多边形）

用户交互：

- 栅栏填充：绘制多边形时自动填充，或通过菜单对已有多边形填充
- 种子填充：选择” 填充 → 种子填充” 菜单，然后点击封闭区域内部任意点
- 可通过颜色对话框选择填充颜色
- 填充操作支持撤销/重做

技术特点：

- 栅栏填充使用 Direct2D 的 PathGeometry，支持 GPU 加速
- 种子填充使用扫描线算法，避免递归栈溢出
- 颜色容差匹配处理 Direct2D 抗锯齿边界
- 使用队列数据结构，效率高于简单递归

3.2.6 Bezier 曲线模块

模块功能：
本模块实现 Bezier 曲线的绘制功能，采用 de Casteljau 递推算法，支持通过多个控制点绘制平滑的参数曲线。Bezier 曲线广泛应用于计算机图形学、CAD 和动画设计中。
主要功能：

- 支持任意数量控制点的 Bezier 曲线绘制（2 个点以上）
- 实时显示控制多边形（灰色虚线）和 Bezier 曲线（蓝色实线）
- 曲线通过起点和终点，受中间控制点影响
- 支持交互式添加控制点
- 曲线平滑连续，无尖角

用户交互：

- 选择” 绘图工具 →Bezier 曲线” 菜单
- 在画布上依次点击添加控制点
- 实时预览控制多边形和曲线效果
- 右键完成曲线绘制
- 可通过移动控制点调整曲线形状

技术特点：

- 使用 de Casteljau 递推算法，数值稳定性好
- 避免高次幂计算，适合高阶 Bezier 曲线
- 默认 100 个采样点，曲线平滑度高
- 几何意义清晰，控制点直观影响曲线形状
- 时间复杂度 $O(n^2 \times samples)$，n 为控制点数

4 实验算法及程序流程

4.1 直线绘制算法

4.1.1 中点画线法

输入：直线起点 (x_0, y_0)，终点 (x_1, y_1)
输出：直线上所有像素点的坐标集合
算法流程（以 $0 < k < 1$ 为例）：

1. 计算 $dx = x_1 - x_0$，$dy = y_1 - y_0$
2. 初始化决策参数 $d = 2dy - dx$
3. 设置起始点 $(x, y) = (x_0, y_0)$，绘制该点
4. 当 $x < x_1$ 时，重复以下步骤：
   (a) 若 $d < 0$，选择 E 点，$d = d + 2dy$，$x = x + 1$
   (b) 若 $d \geq 0$，选择 NE 点，$d = d + 2dy - 2dx$，$x = x + 1$，$y = y + 1$
   (c) 绘制点 (x, y)
5. 算法结束

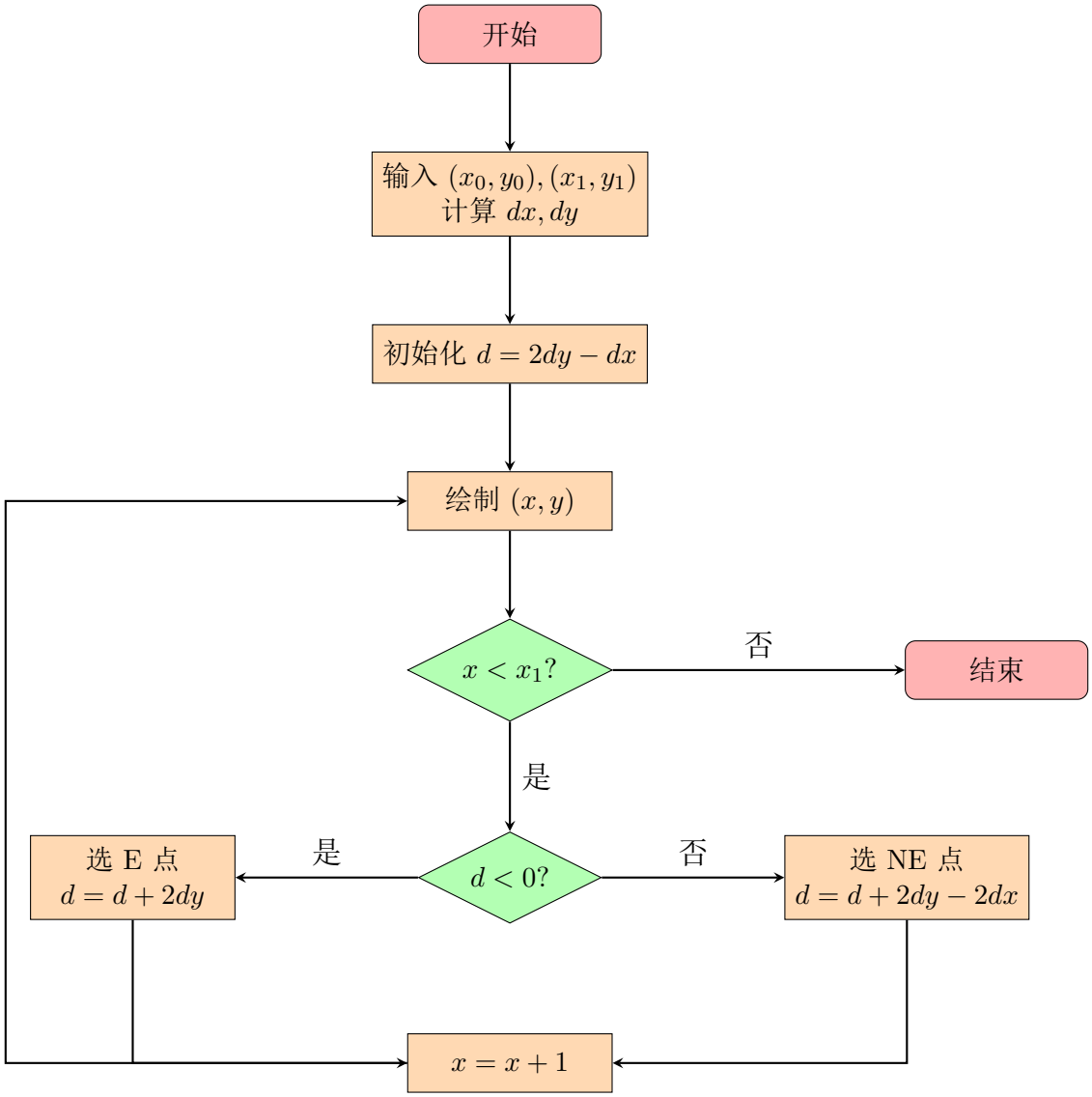


图 2: 中点画线法流程图

4.1.2 Bresenham 画线法

输入：直线起点 (x_0, y_0)，终点 (x_1, y_1)
输出：直线上所有像素点的坐标集合
算法流程：

1. 计算 $dx = |x_1 - x_0|$，$dy = |y_1 - y_0|$
2. 确定步进方向 $sx = (x_0 < x_1)?1 : -1$，$sy = (y_0 < y_1)?1 : -1$
3. 初始化误差项 $err = dx - dy$
4. 设置当前点 $(x, y) = (x_0, y_0)$

5. 循环执行直到 $(x, y) = (x_1, y_1)$：

   (a) 绘制点 (x, y)

   (b) 计算 $e2 = 2 \times err$

   (c) 若 $e2 > -dy$，则 $err = err - dy$，$x = x + sx$

   (d) 若 $e2 < dx$，则 $err = err + dx$，$y = y + sy$

6. 算法结束

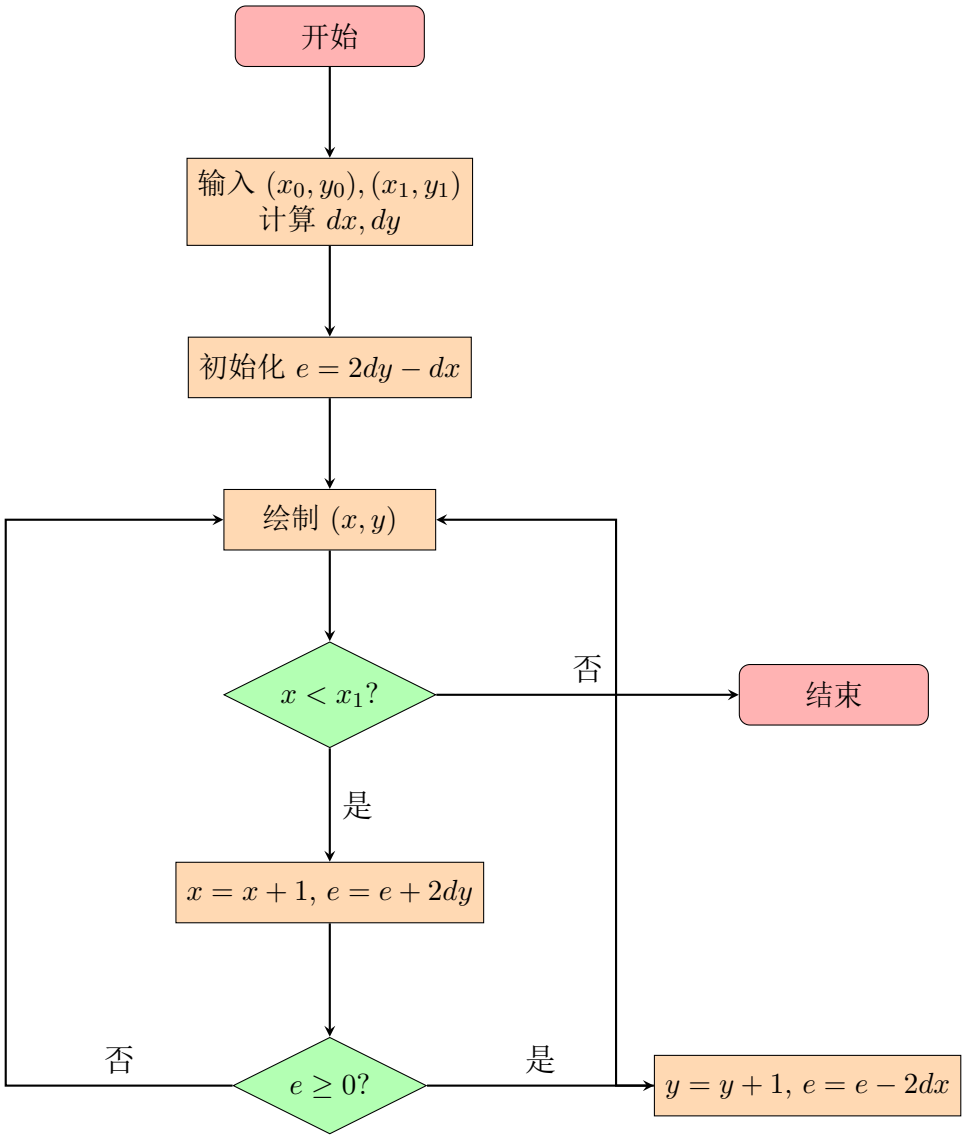


图 3: Bresenham 画线法流程图

4.2 圆绘制算法

4.2.1 中点画圆法

输入：圆心坐标 (x_c, y_c)，半径 r
输出：圆上所有像素点的坐标集合
算法流程：

1. 初始化 $x = 0$，$y = r$，$d = 1 - r$
2. 利用八对称性绘制 8 个对称点：$(x_c \pm x, y_c \pm y)$ 和 $(x_c \pm y, y_c \pm x)$
3. 当 $x < y$ 时，重复以下步骤：
   (a) 若 $d < 0$，选择 E 点，$d = d + 2x + 3$
   (b) 若 $d \geq 0$，选择 SE 点，$d = d + 2(x - y) + 5$，$y = y - 1$
   (c) $x = x + 1$
   (d) 绘制 8 个对称点
4. 算法结束

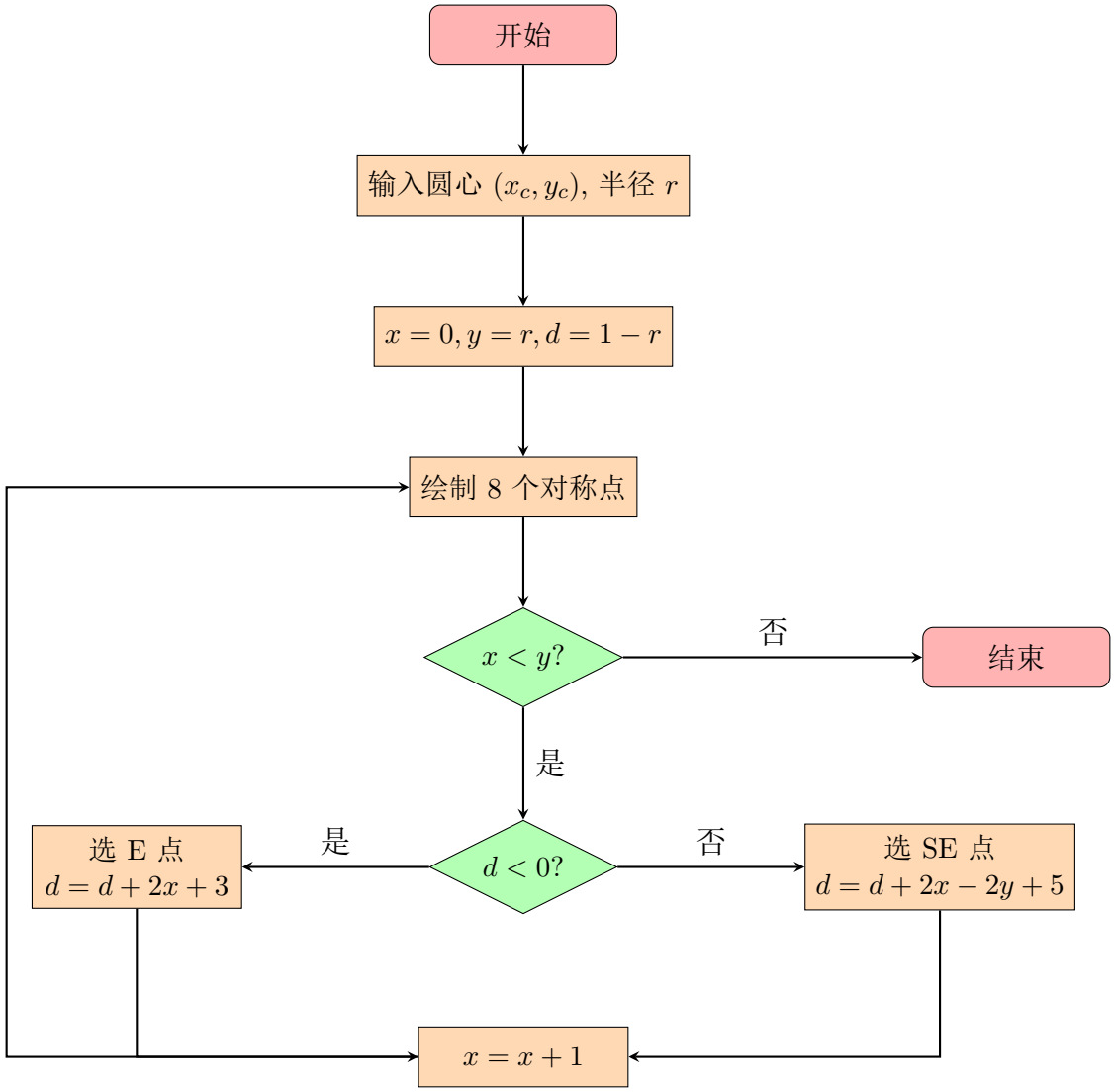


图 4: 中点画圆法流程图

4.2.2 Bresenham 画圆法

输入：圆心坐标 (x_c, y_c)，半径 r
输出：圆上所有像素点的坐标集合
算法流程：

1. 初始化 $x = 0$，$y = r$，$d = 3 - 2r$
2. 利用八对称性绘制 8 个对称点
3. 当 $x \leq y$ 时，重复以下步骤：
   (a) 若 $d < 0$，选择 E 点，$d = d + 4x + 6$

(b) 若 $d \geq 0$，选择 SE 点，$y = y - 1$，$d = d + 4(x - y) + 10$

(c) $x = x + 1$

(d) 若 $x \leq y$，绘制 8 个对称点

4. 算法结束

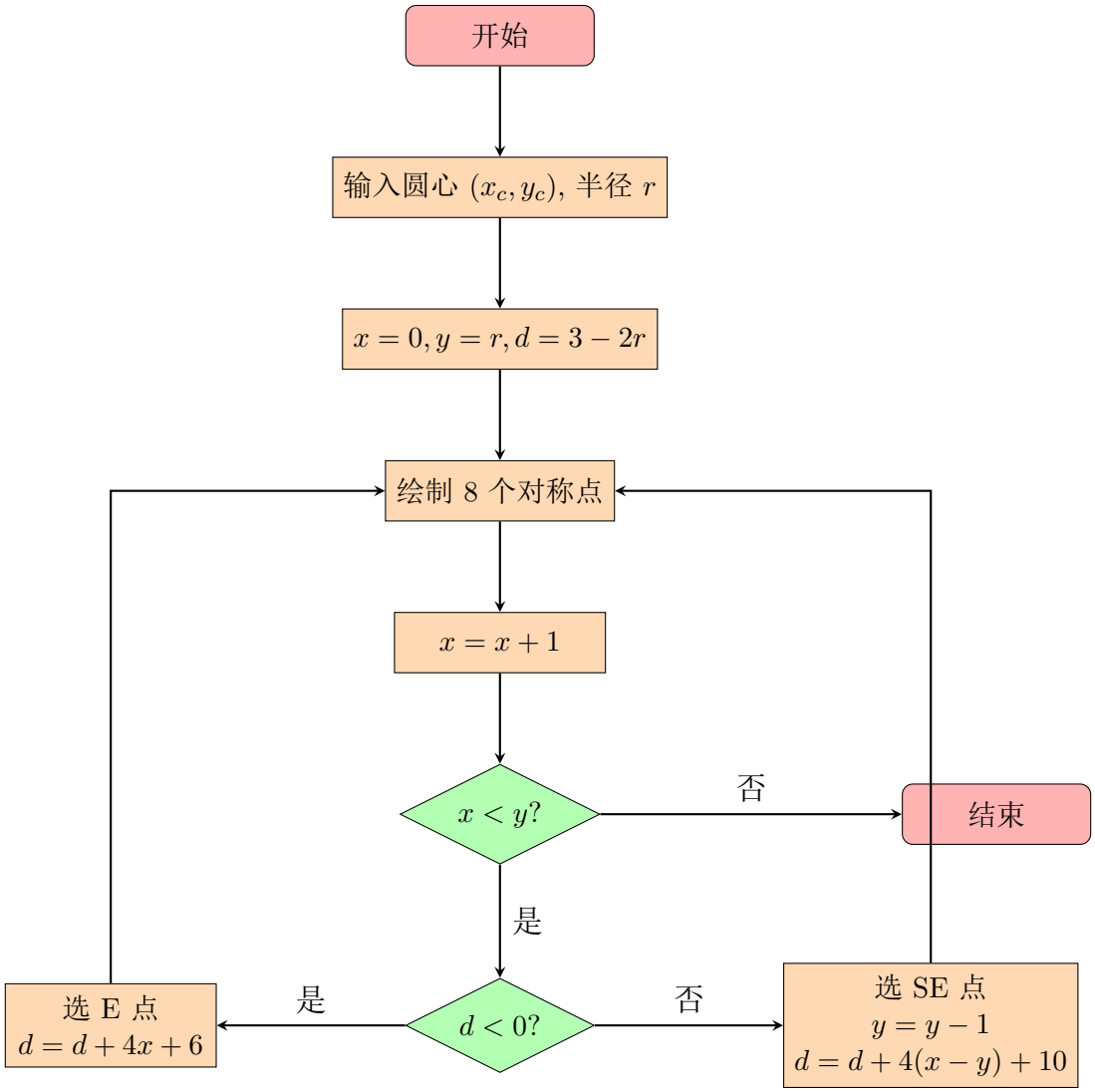


图 5: Bresenham 画圆法流程图

4.3 种子填充算法

输入：种子点坐标 (x_{seed}, y_{seed})，填充颜色 $fillColor$，目标颜色 $targetColor$

输出：填充后的图像
算法流程：

1. 获取种子点的颜色作为目标颜色 $targetColor$
2. 从种子点向左右扩展，找到当前扫描线上的可填充区间 $[x_l, x_r]$
3. 填充区间 $[x_l, x_r]$ 内的所有像素
4. 将区间信息 $\{x_l, x_r, y, +1\}$ 和 $\{x_l, x_r, y, -1\}$ 加入队列
5. 当队列不为空时，重复以下步骤：
   (a) 从队列取出一个区间 $\{x_l, x_r, y, dy\}$
   (b) 计算相邻扫描线 $y' = y + dy$
   (c) 在 $[x_l, x_r]$ 范围内扫描 y' 行，找到所有可填充区间
   (d) 填充找到的区间，并将新区间加入队列
6. 算法结束

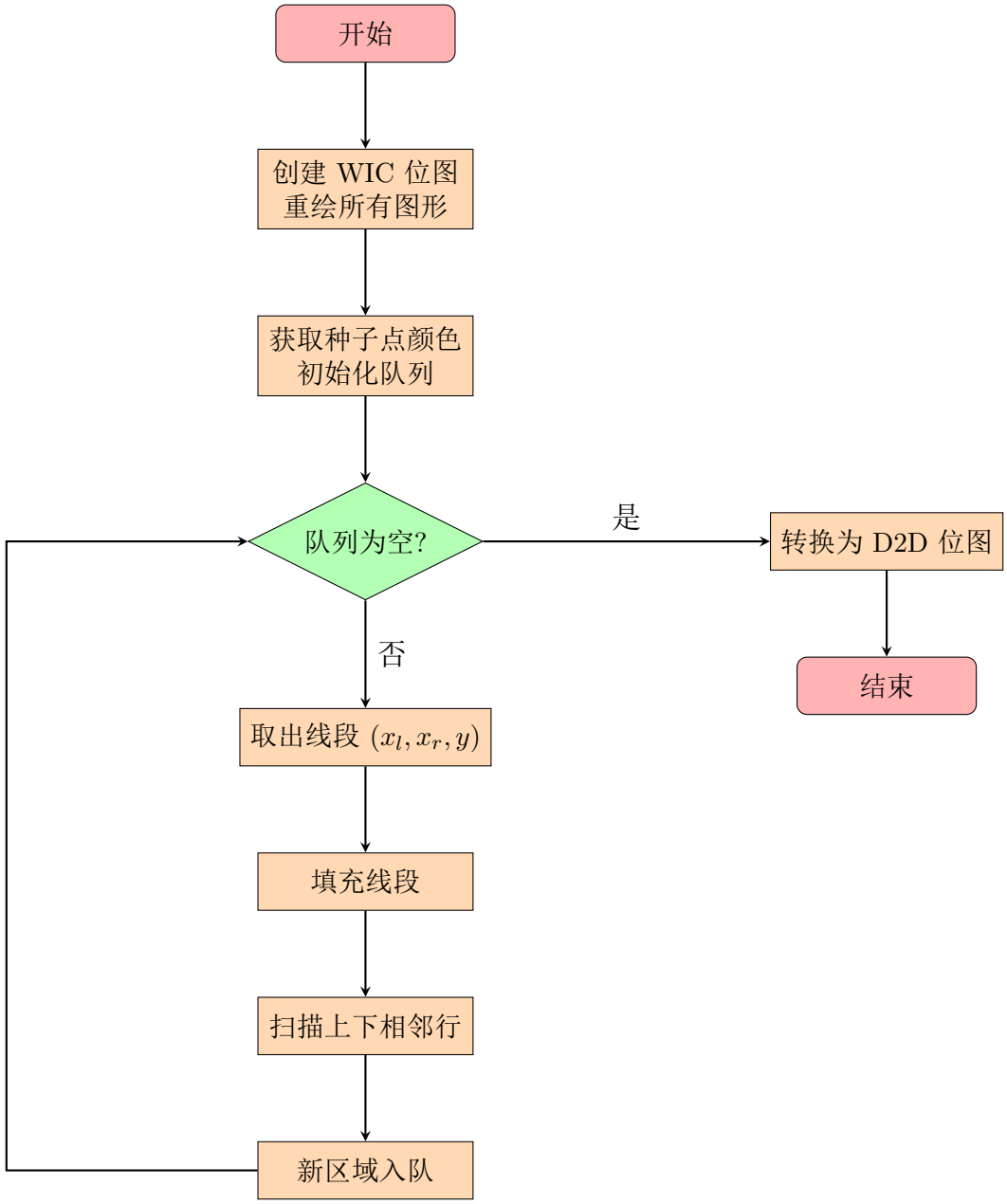


图 6: 种子填充算法流程图

4.4 扫描线填充算法

输入：多边形顶点数组 $\{P_0, P_1, \ldots, P_{n-1}\}$，填充颜色 $fillColor$
输出：填充后的多边形
算法流程：

1. 计算多边形的 y 坐标范围 $[y_{min}, y_{max}]$
2. 对于每条扫描线 y（从 y_{min} 到 y_{max}）：

(a) 计算扫描线与多边形各边的交点

(b) 将所有交点按 x 坐标排序

(c) 按奇偶规则填充：第 1-2 个交点之间填充，第 3-4 个交点之间填充，依此类推

3. 算法结束

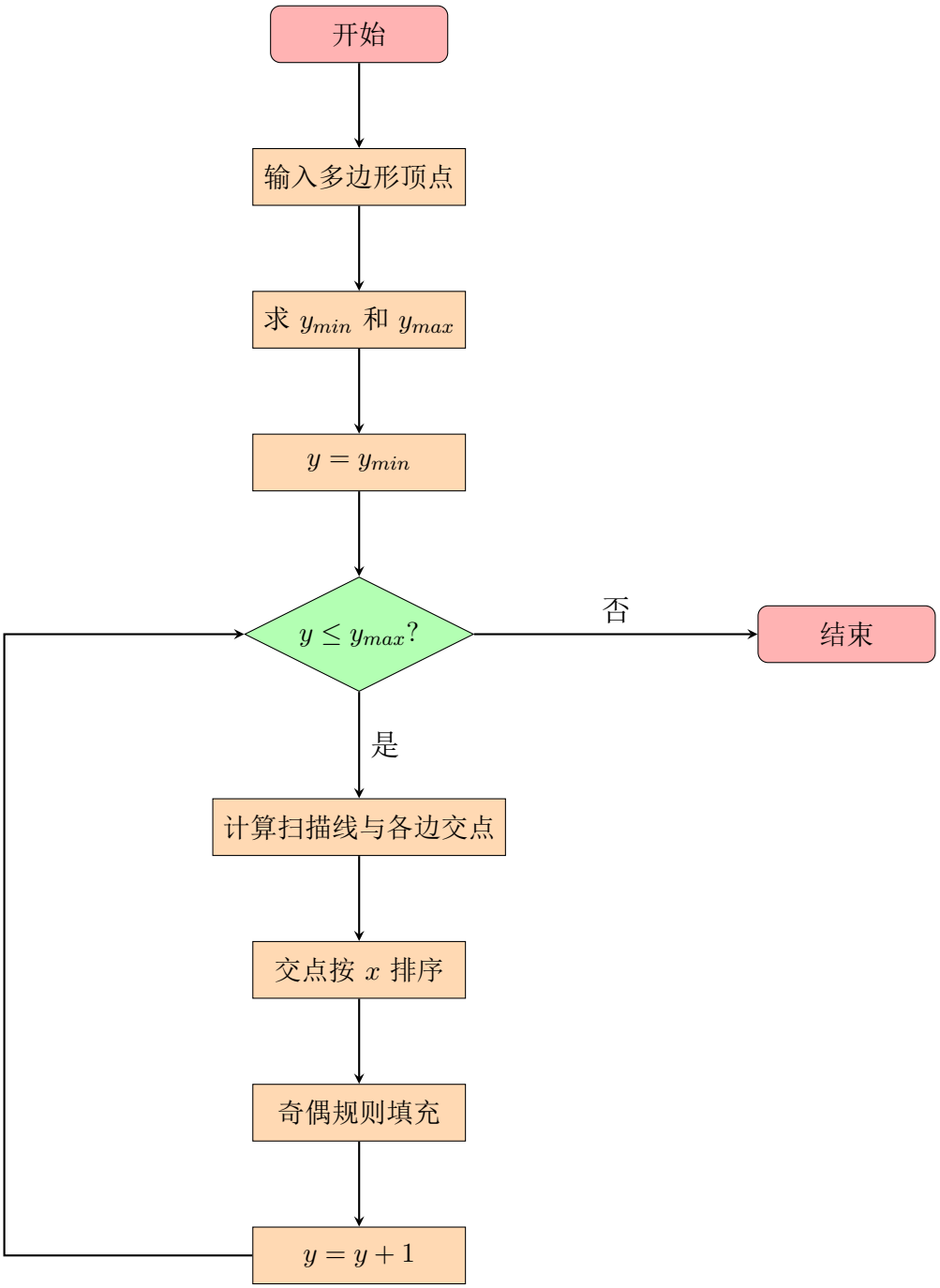


图 7: 扫描线填充多边形流程图

4.5 粗线绘制算法

输入：线段起点 (x_1, y_1)，终点 (x_2, y_2)，线宽 $width$
输出：指定宽度的粗线
算法流程：

1. 若 $width \leq 1$，直接使用细线绘制算法，结束
2. 计算线段方向向量 $\vec{V} = (dx, dy) = (x_2 - x_1, y_2 - y_1)$
3. 计算线段长度 $len = \sqrt{dx^2 + dy^2}$
4. 计算垂直方向的单位向量 $\vec{V}_\perp = (-dy/len, dx/len)$
5. 计算偏移量 $offset = \vec{V}_\perp \times (width/2)$
6. 计算矩形 4 个顶点：$v_0 = (x_1, y_1) + offset$，$v_1 = (x_1, y_1) - offset$，$v_2 = (x_2, y_2) - offset$，$v_3 = (x_2, y_2) + offset$
7. 调用 FillPolygon 填充矩形
8. 算法结束

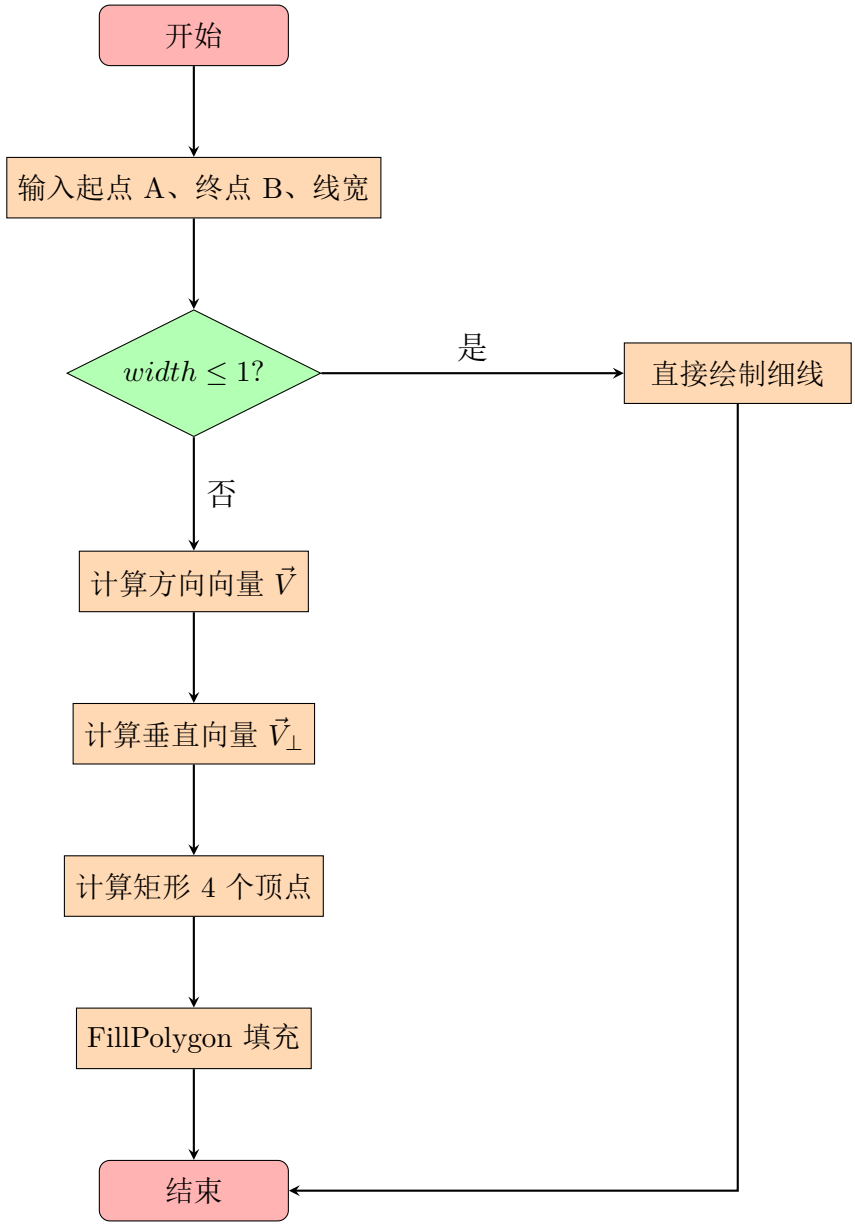


图 8: 粗线绘制流程图

4.6 粗圆绘制算法

输入：圆心 (x_c, y_c)，半径 r，线宽 $width$
输出：指定宽度的粗圆
算法流程：

1. 若 $width \leq 1$，直接使用细圆绘制算法，结束
2. 计算外圆半径 $r_{outer} = r + width/2$

3. 计算内圆半径 $r_{inner} = r - width/2$（若 $r_{inner} < 0$ 则设为 0）

4. 创建 PathGeometry 路径对象

5. 绘制外圆（顺时针方向）

6. 若 $r_{inner} > 0$，绘制内圆（逆时针方向，形成环形）

7. 调用 FillGeometry 填充环形区域

8. 算法结束

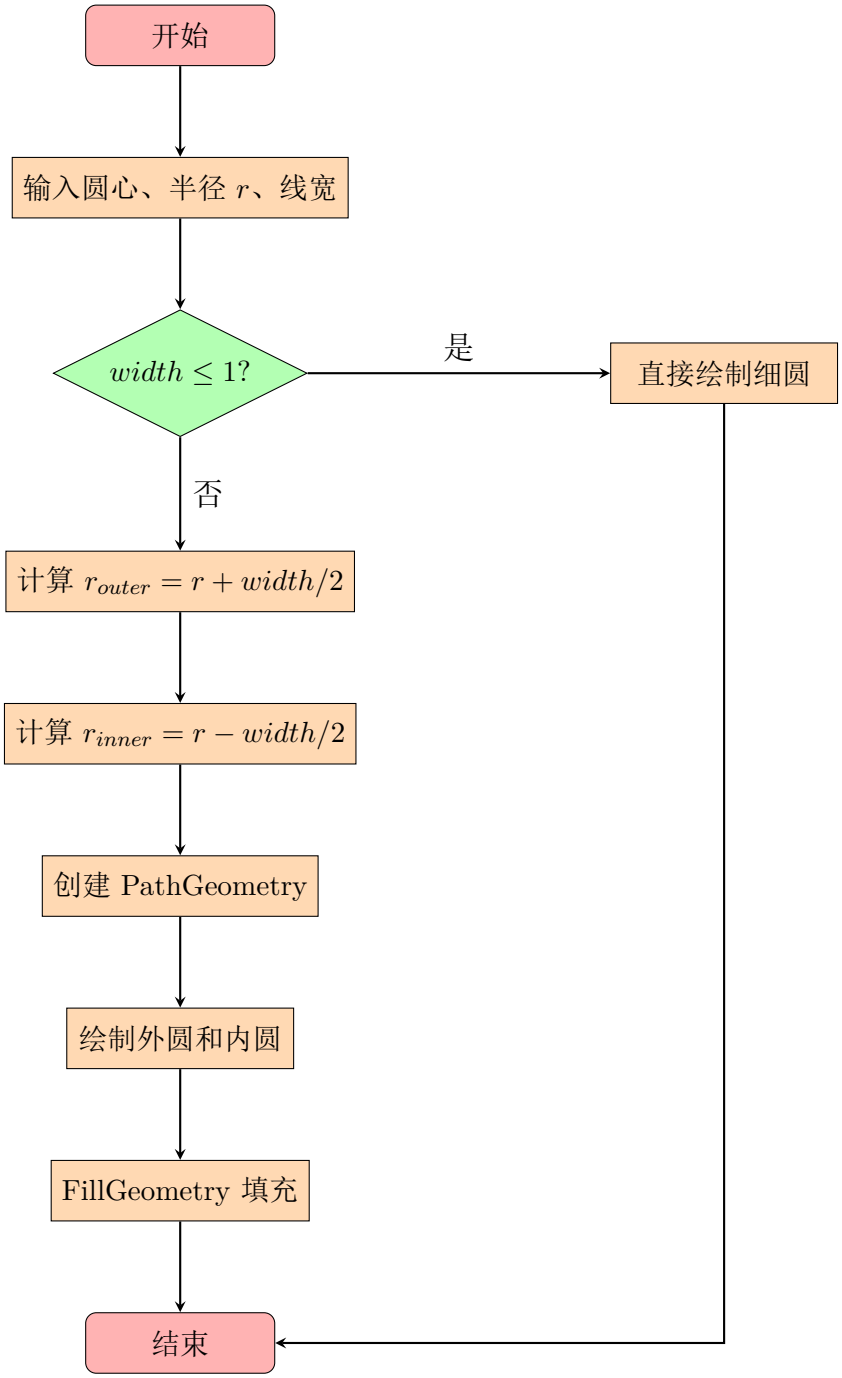

开始
输入圆心、半径 r、线宽
$width \leq 1$?
是
直接绘制细圆
否
计算 $r_{outer} = r + width/2$
计算 $r_{inner} = r - width/2$
创建 PathGeometry
绘制外圆和内圆
FillGeometry 填充
结束

图 9: 粗圆绘制流程图

4.7 Bezier 曲线算法

4.7.1 de Casteljau 递推算法

输入：n 个控制点 $\{P_0, P_1, \ldots, P_{n-1}\}$，参数 $t \in [0,1]$
输出：Bezier 曲线上参数 t 对应的点 $B(t)$
算法流程：

1. 复制控制点数组到临时数组 $points$
2. 对于递推层次 r 从 1 到 $n-1$：
   (a) 对于索引 i 从 0 到 $n-r-1$：
      i. 计算 $points[i] = (1-t) \times points[i] + t \times points[i+1]$
3. 返回 $points[0]$ 作为曲线上的点 $B(t)$
4. 算法结束

递推公式：$P_i^r(t) = (1-t) \cdot P_i^{r-1}(t) + t \cdot P_{i+1}^{r-1}(t)$，最终结果 $B(t) = P_0^{n-1}(t)$

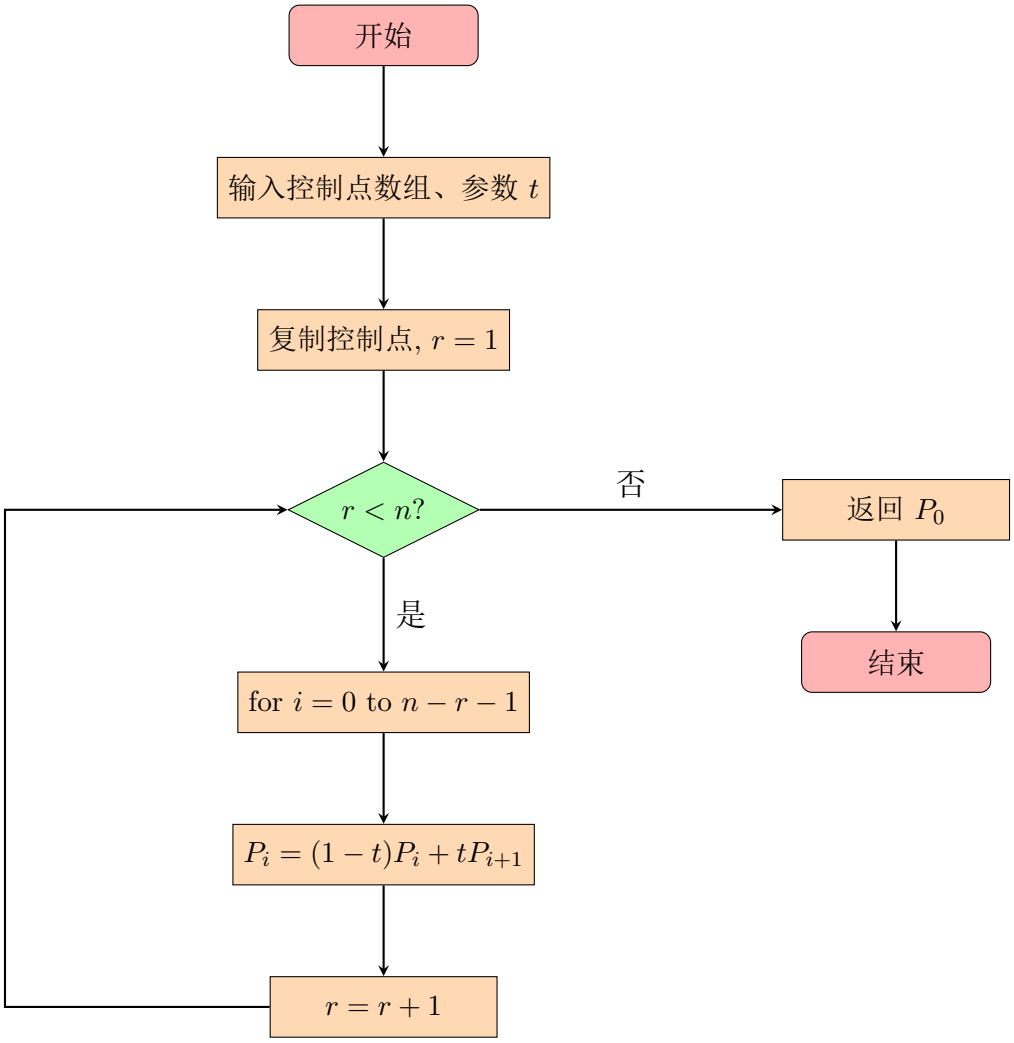


图 10: de Casteljau 递推算法流程图

4.7.2 Bezier 曲线生成流程

输入：n 个控制点 $\{P_0, P_1, \ldots, P_{n-1}\}$，采样数 $samples$
输出：Bezier 曲线的绘制
算法流程：

1. 初始化曲线点数组 $curvePoints$
2. 对于 i 从 0 到 $samples$：
   (a) 计算参数 $t = i/samples$
   (b) 调用 de Casteljau 算法计算 $B(t)$
   (c) 将 $B(t)$ 添加到 $curvePoints$

3. 用直线段连接 $curvePoints$ 中相邻的点，绘制曲线

4. 算法结束

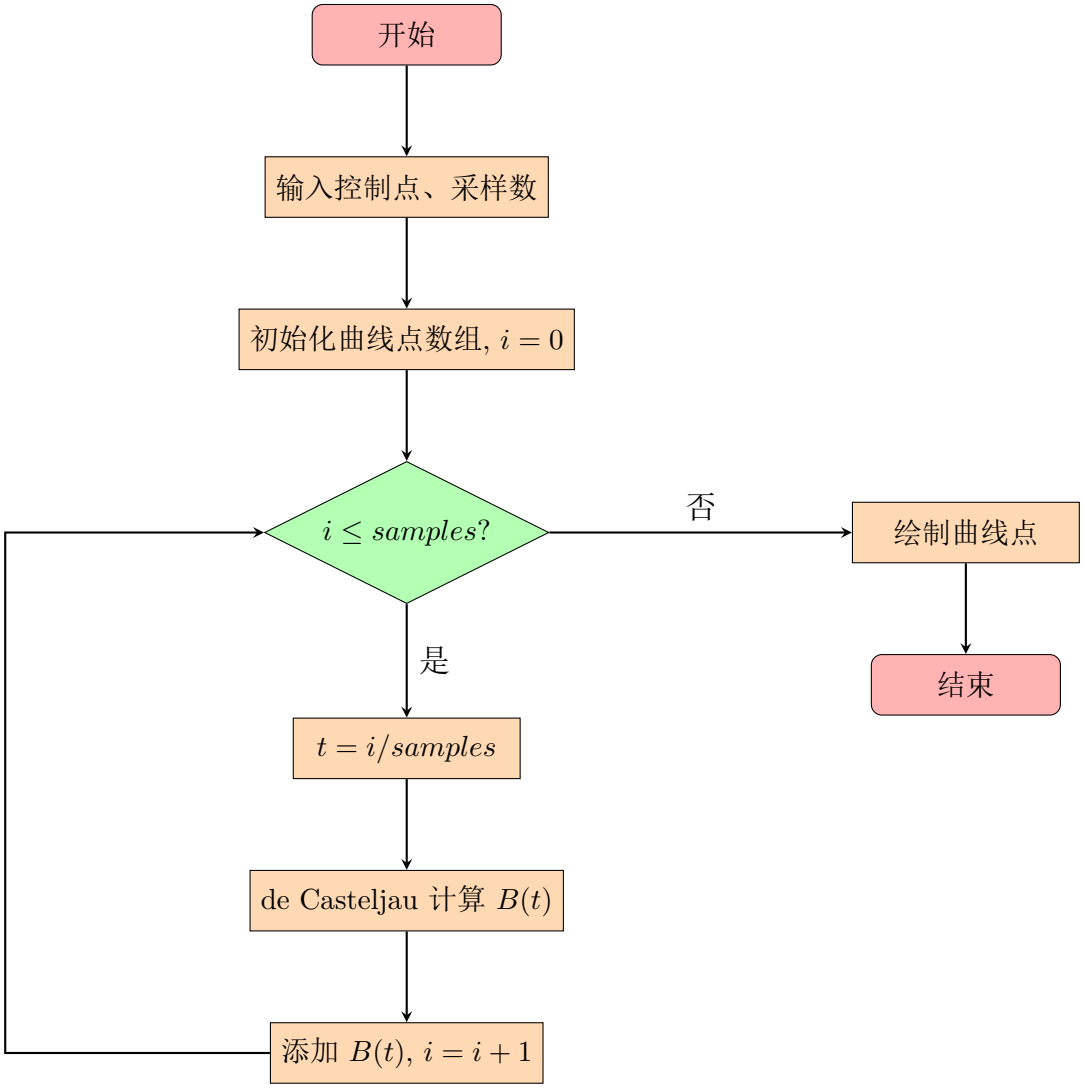


图 11: Bezier 曲线生成流程图

5 实验结果

本实验成功实现了以下功能：

1. 直线绘制：实现了中点画线法和 Bresenham 画线法，支持任意斜率的直线绘制，通过分 8 个象限处理实现了完整的直线绘制功能
2. 圆绘制：实现了中点画圆法和 Bresenham 画圆法，利用八对称性只需计算 1/8 圆弧即可得到完整的圆，大大提高了绘制效率
3. 线型控制：支持实线、虚线、点线、点划线四种线型，通过 Direct2D 的 StrokeStyle 实现
4. 线宽控制：使用扫描线填充法实现粗线和粗圆的绘制，粗线转换为矩形多边形填充，粗圆转换为环形填充
5. 区域填充：实现了种子填充算法和扫描线填充算法，种子填充适合任意形状的封闭区域，扫描线填充适合已知顶点的多边形
6. **Bezier** 曲线：使用 de Casteljau 递推算法绘制任意阶数的 Bezier 曲线，通过对参数 t 进行采样得到曲线上的点

6 实验总结

通过本次实验，深入理解了计算机图形学中基本图元生成算法的原理和实现方法：

1. 中点画线法和 **Bresenham** 算法：两种算法本质相同，都是通过整数运算避免浮点计算，提高了绘制效率。中点法从几何角度推导，Bresenham 法从误差累积角度推导。
2. 圆的绘制：利用圆的八对称性，只需计算 1/8 圆弧即可得到完整的圆。中点画圆法和 Bresenham 画圆法的决策参数不同，但都能高效地绘制圆。
3. 区域填充：种子填充算法适合填充任意形状的封闭区域，但需要访问像素数据；扫描线填充算法适合已知顶点的多边形，可以利用 GPU 加速。
4. **Bezier** 曲线：de Casteljau 算法具有数值稳定性好、几何意义清晰的优点，通过递推计算可以得到曲线上任意参数对应的点。
5. 粗线绘制：将线段转换为矩形多边形，使用扫描线填充法实现，这种方法可以绘制任意宽度的线段。

本实验加深了对计算机图形学基础算法的理解，提高了使用 Direct2D 进行图形编程的能力。